

Tendências de Motores de Jogos

Semana 14

Packaging e Playtest Cruzado — Encerramento do Módulo 4

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 4 — Produzir como um Pequeno Estúdio

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender o que diferencia um protótipo rodando no editor de um build distribuível
- Compreender as configurações de Packaging e a escolha de plataforma-alvo como decisões técnicas, não apenas passos operacionais
- Empacotar o Vertical Slice de cada grupo em um build estável, executável fora do editor
- Validar o build de um colega via Playtest cruzado e conduzir o Code Review de encerramento do Módulo 4

Como a semana está organizada



Encontro 1

Pipeline de Packaging — do protótipo no editor ao build distribuível



Encontro 2

Playtest cruzado + Code Review de encerramento do Módulo 4

Tendências de Motores de Jogos

ENCONTRO 1

Pipeline de Packaging

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



O jogador final vai abrir o editor da Unreal para jogar seu projeto?

Pense no que precisa acontecer entre "o jogo roda no meu editor" e "qualquer pessoa consegue jogar sem instalar nada além do jogo".

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Separar o ambiente de edição do produto final

- Todo motor de jogo separa o editor (iteração rápida) do build (produto que o jogador executa)
- Packaging compila o projeto, empacota os assets referenciados e gera um executável autônomo para uma plataforma-alvo
- O processo expõe problemas invisíveis no editor: referências quebradas, assets não incluídos, dependências ausentes
- Por isso Packaging não é um passo burocrático de final de produção — é o momento em que o projeto enfrenta as condições reais do jogador

DICA

Um projeto com gargalos não tratados na Semana 13 chega ao build do mesmo jeito — só que mais difícil de diagnosticar fora do editor.

O que muda entre Play In Editor e o build empacotado

- Play In Editor carrega o projeto com todo o ambiente de desenvolvimento por trás
- O build empacotado depende exclusivamente do que foi explicitamente incluído no pacote
- Configurações relevantes: plataforma-alvo, configuração de build (Development/Shipping), mapa inicial (Maps & Modes)
- Espaço em disco e tempo de build variam com o tamanho do projeto — devem ser estimados antes da aula

Packaging: Unreal × Unity

Unreal

Project Settings concentra as configurações de Packaging (plataforma, build, mapa inicial) antes de uma única ação de empacotamento

Unity

Build Settings expõe plataforma-alvo e configuração de build (Debug/Release) diretamente na janela, a cada execução

Do projeto editável ao build distribuível



Demonstração: configuração e empacotamento guiado

O professor configura plataforma-alvo, build (Development/Shipping) e mapa inicial em um projeto de teste, executa o Packaging completo e mostra o build resultante rodando fora do editor.

Resultado esperado: um executável autônomo, gerado a partir das mesmas configurações que cada grupo aplicará ao próprio Vertical Slice.

Imagem sugerida

Objetivo: mostrar a transição entre o projeto rodando no editor e o mesmo projeto rodando como build empacotado.

Enquadramento: duas capturas de tela lado a lado — a Unreal Engine em Play In Editor à esquerda, o executável empacotado rodando em janela própria fora do editor à direita.

Elementos presentes: interface do editor com o nível carregado; janela do build empacotado com o mesmo nível, sem nenhum elemento de editor visível.

Destaque visual: a ausência total da interface do editor no build da direita.

Legenda sugerida: "O jogador nunca vê o editor — apenas o build."

Boas práticas

✓ BOAS PRÁTICAS

- Testar o Packaging cedo, nunca apenas no fim do prazo — problemas de referência só aparecem no build
- Definir o mapa inicial (Maps & Modes) explicitamente, nunca depender do último nível aberto no editor
- Isolar assets de teste/temporários fora da estrutura final antes de empacotar, conforme PROJECT_ARCHITECTURE.md

Laboratório do Encontro 1

Cada grupo empacota o próprio Vertical Slice, já otimizado desde a Semana 13, resolvendo problemas de referência ou conteúdo ausente que surjam no processo.

OBJETIVOS

Critério de sucesso: build empacotado do próprio Vertical Slice, executável fora do editor, sem erros de referência ou conteúdo ausente, preservando intactos todos os sistemas construídos até a Semana 13.

Fechando o Encontro 1

- Packaging separa o ambiente de edição do produto que o jogador realmente executa
- O build expõe problemas invisíveis no editor — por isso depende do profiling já feito na Semana 13
- Cada grupo já possui um build estável, pronto para o Playtest cruzado do Encontro 2

Tendências de Motores de Jogos

ENCONTRO 2

Playtest Cruzado e Code Review de Encerramento

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Quem enxerga melhor os problemas do seu projeto: sua equipe ou um jogador de fora?

Pense em algo que você já testou tantas vezes que parou de perceber como um problema.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

A perspectiva que a própria equipe não tem

- Nenhuma equipe de desenvolvimento avalia objetivamente o próprio produto — confusões e bugs ficam invisíveis para quem construiu o sistema
- O Playtest cruzado formaliza a perspectiva externa: cada grupo passa a ocupar, por um momento, o papel de jogador final de outro grupo
- Essa prática é universal a qualquer produção de jogos, independentemente da engine utilizada
- Antecipa o papel do Code Review de encerramento: avaliar o produto entregável como um todo, não sistemas isolados

DICA

O Code Review desta semana não pergunta mais "este sistema funciona?" — pergunta "este produto está pronto para alguém de fora jogar?"

Como o rodízio funciona

- O professor organiza previamente o esquema de rodízio — nenhum grupo testa o próprio build
- Cada grupo executa o build empacotado de outro grupo e registra bugs e pontos de confusão observados
- O registro recebido é usado pelo próprio grupo para tratar os problemas mais críticos dentro do tempo disponível
- Não há lista fechada de correções — cada grupo prioriza o que compromete jogabilidade ou estabilidade sobre ajustes cosméticos

Validação de build: Unreal × Unity

Unreal

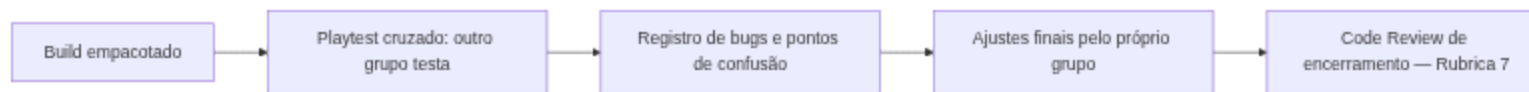
Build empacotado testado fora do editor, revisão de conteúdo via Content Browser durante o Code Review

Unity

Processos equivalentes de build testing e revisão de projeto antes de um marco de entrega, sem ferramenta dedicada distinta

Tendências de Motores de Jogos

Do build ao ajuste final



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Demonstração: como registrar um bug ou ponto de confusão

O professor demonstra, sobre um build de teste, como diferenciar um bug (algo que quebra) de um ponto de confusão (algo que funciona mas não é claro para quem joga pela primeira vez), usando um modelo simples de registro.

Resultado esperado: cada grupo sai do Playtest cruzado com um registro claro, priorizável por impacto, do build que testou.

Imagem sugerida

Objetivo: ilustrar o momento do Playtest cruzado, com um grupo jogando o build de outro grupo.

Enquadramento: foto ou captura de tela do build empacotado rodando em uma máquina, com um formulário/checklist de registro de bugs visível ao lado.

Elementos presentes: janela do build fora do editor; checklist com colunas "bug" e "ponto de confusão"; anotação de exemplo preenchida.

Destaque visual: a separação visual entre "bug" e "ponto de confusão" no checklist.

Legenda sugerida: "Registrar o que confunde é tão valioso quanto registrar o que quebra."

Boas práticas

✓ BOAS PRÁTICAS

- Priorizar por impacto (jogabilidade/estabilidade) antes de iniciar qualquer correção, nunca corrigir por ordem de registro
- Tratar o registro do Playtest cruzado como dado de produção, não como crítica pessoal ao grupo
- Documentar as decisões técnicas do projeto de forma que alguém de fora consiga entendê-las sem o grupo presente

Laboratório/Desafio: Playtest cruzado e ajustes finais

Cada grupo testa o build empacotado de outro grupo, registra bugs e pontos de confusão, e depois trata os problemas mais críticos apontados no próprio build, priorizando por impacto.

OBJETIVOS

Critério de sucesso: build de outro grupo testado com registro produzido; problemas mais críticos do próprio build tratados dentro do tempo disponível, sem regressão em nenhum sistema já construído.

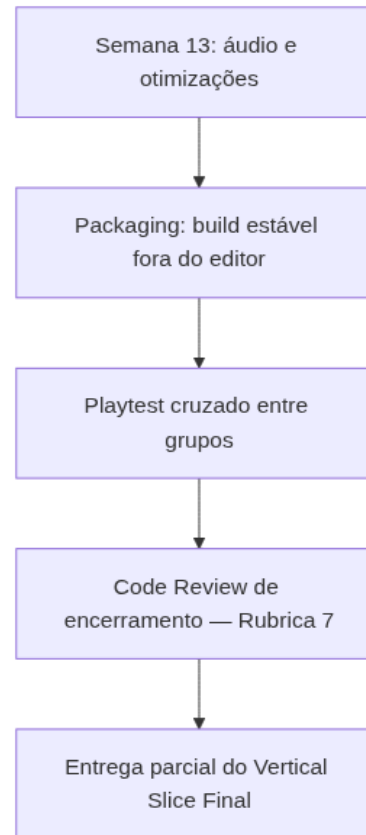
Code Review de Encerramento do Módulo 4 — Rubrica 7

Avaliação formal do Vertical Slice como produto entregável: arquitetura, gameplay, organização, qualidade técnica, polimento, uso correto dos recursos da Unreal, consistência, documentação, empacotamento e capacidade de explicar decisões.

ATENÇÃO

Esta é a entrega parcial da Rubrica 7 (Vertical Slice Final), reconfirmada na Semana 17 — não avaliar qualidade artística nem quantidade de assets.

Onde a Semana 14 se encaixa no Vertical Slice



Resumo da Semana 14

- Packaging separa o ambiente de edição do produto que o jogador realmente executa
- A perspectiva externa de um jogador é insubstituível na revisão de um produto — nenhuma equipe se avalia objetivamente
- Cada grupo possui um build estável, testado por outro grupo, com ajustes críticos tratados
- Code Review de encerramento do Módulo 4 concluído (Rubrica 7 — entrega parcial)

Checklist Final do Encontro

OBJETIVOS

- Build do Vertical Slice empacotado, executável fora do editor, sem erros de referência
- Build testado via Playtest cruzado por outro grupo, com registro de bugs e pontos de confusão
- Problemas mais críticos do próprio build tratados dentro do tempo disponível
- Todos os sistemas dos Módulos 1 a 4 intactos no build final
- Code Review de encerramento do Módulo 4 concluído (Rubrica 7)

Próximos passos



DICA

A Semana 15 abre a Unidade V — Comparar Arquiteturas, com metodologia de Reverse Engineering e autonomia muito alta: os estudantes passam a analisar projetos profissionais (Lyra, Stack O Bot, Content Examples) em vez de continuar construindo o próprio Vertical Slice. O build estável desta semana serve como ponto de comparação concreto entre as decisões arquiteturais de cada grupo e as decisões observadas nos projetos profissionais.

Leitura recomendada: Unreal Engine — Packaging Your Project (Epic Games Documentation).